



Combining SetFit and LLMs for Explainable Security Bug Report Pre- diction

Hao Pan

This thesis is submitted to the Faculty of Engineering at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Software Engineering. The thesis is equivalent to 20 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author(s):

Hao Pan

E-mail: hapa24@student.bth.se

University advisor:

Dr. Muhammad Laiq

Department of Software Engineering

Faculty of Engineering
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Security bug reports (SBRs) describe software defects with potential security implications and require timely identification during bug triage. However, SBR prediction is challenging because security-related reports are rare, imbalanced, and sometimes mislabeled as ordinary bug reports. This thesis investigates an explainable SBR prediction pipeline that combines SetFit-based classification, post-hoc feature attribution (LIME&SHAP), and LLM-assisted explanation generation. The results show that SetFit performance varies across datasets. The faithfulness evaluation indicates that LIME provides more faithful explanations than SHAP in this experimental setting, achieving stronger removal-based effects and better rationale sufficiency across the studied datasets. The LLM-assisted explanation stage demonstrates that attribution evidence can be converted into readable narratives. Overall, the thesis shows that explainable SBR prediction requires both technical faithfulness evaluation and human-oriented explanation assessment.

Keywords: SetFit, XAI, SBRs, explainability

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Problem Statement and Scope	2
1.3 Aim, Objectives and Research Questions	2
1.4 Contributions	3
1.5 Thesis Outline	3
2 Related Work	5
2.1 Security Bug Report Prediction	5
2.2 Few-shot Learning and SetFit for SBR Prediction	6
2.3 Explainable AI for Text Classification and LIME & SHAP	6
2.4 Faithfulness, Plausibility, and Explanation Evaluation	7
2.5 LLM-assisted Explanation Generation	8
2.6 Summary and Research Gap	8
3 Method	11
3.1 Research Design	11
3.2 Research Questions and Evaluation Strategy	11
3.3 Dataset Selection and Preprocessing	13
3.4 SetFit-based SBR classification	14
3.5 Post-hoc Explanation with LIME and SHAP	14
3.6 Faithfulness Evaluation Metrics	14
3.7 LLM-based Explanation Generation	16
3.8 User Study	16
3.9 Data Analysis Procedure	16
3.10 Validity and Reliability Considerations	17
4 Results and Analysis	19
4.1 Overview of the Results	19
4.2 Dataset and Preprocessing Results	19
4.3 SetFit Classification Results	19
4.4 LIME and SHAP Explanation Results	19
4.5 Faithfulness Evaluation Results	20
4.6 LLM-assisted Explanation Results	21
4.7 Human Evaluation Results	22
4.8 Summary of Findings	22

5	Discussion	23
5.1	Overview of the Main Findings	23
5.2	Faithfulness of LIME and SHAP	23
5.3	LLM-generated Explanations and Human Perception	24
5.4	Implications for Research and Practice	24
5.5	Threats to Validity	25
5.6	Broader Discussion	25
6	Conclusions and Future Work	27
6.1	Conclusion	27
6.2	Future Work	27
	References	29
A	Supplemental Information	31
A.1	AI Declaration	31
A.2	User Study Questionnaire	31
A.3	LLM Prompt	31

1.1 Background

Software maintenance plays an important role in the Software Development Life Cycle (SDLC), as it accounts for a major share of a software system’s total life cycle cost [5]. In software maintenance, timely bug location is crucial because such defects can lead to software failures and losses. Thus, bug-tracking systems (BTS) such as Jira¹, Bugzilla², and GitHub Issues³ are used to record and manage software defects. However, creating valuable bug reports is often challenging because it relies heavily on complete information, which significantly slows the bug resolution process [18].

Among the different types of bug reports, security bug reports (SBRs) constitute a very small, severely imbalanced class [13] but describe critical vulnerabilities in software products. Delay in identifying and fixing the security bugs involved in the SBRs can cause serious damage to software system stakeholders. However, bug reporters may often mislabel SBRs as other bug reports due to a lack of security domain knowledge [7].

Traditionally, identifying SBRs has relied heavily on manual triage by security engineers reading text to judge, which is time-consuming and often infeasible [7]. Relying solely on human effort in large-scale repositories leads to misclassification and delayed responses to critical threats. This motivates the development of automated SBR prediction techniques to support practitioners in prioritizing security-relevant reports.

While automated classification systems [21], such as SetFit, a few-shot learning model that fits imbalanced scenarios [10], have shown remarkable potential in identifying such vulnerabilities, existing work focuses solely on model performance. However, explainability and interpretability are equally important. Their black box nature hinders their adoption in practice [12]. Developers and security analysts need to understand why a model flags a particular issue as a security flaw, a need that existing explainable artificial intelligence (xAI) frameworks [4] attempt to address through a variety of technical outputs. However, these technical representations often require expert interpretation to be actionable. Recent work indicates that LLMs are good at explaining AI narratives as a generator rather than an explainer [16], suggesting potential applications in the security domain.

¹<https://www.atlassian.com/software/jira>

²<https://www.bugzilla.org>

³<https://github.com/features/issues>

1.2 Problem Statement and Scope

Although automated classifiers can reduce the effort required for SBR identification, many provide limited insight into their decision-making processes. This is a problem in the security domain, where false negatives may delay vulnerability mitigation, and false positives may waste expert effort. Without explanations, developers cannot determine whether a prediction is based on meaningful evidence or just simply on text.

Post-hoc explanation techniques such as Local Interpretable Model-agnostic Explanations (LIME) [14] and SHapley Additive exPlanations (SHAP) [11] can identify influential tokens behind a model prediction. However, their outputs are often presented as weighted keyword lists, which may be difficult for developers to understand. LLMs offer a possible way to transform these technical explanations into natural language. Yet, a fluent explanation may be plausible to humans without faithfully representing the model's true meaning.

Therefore, the scope of this thesis is limited to explainable SBR prediction using SetFit on four open-source datasets. Ambari ⁴, Camel ⁵, Derby ⁶, and Wicket ⁷. The study focuses on two post-hoc explanation methods, LIME and SHAP, and uses an LLM only as a narrative generator. The evaluation, therefore, considers both objective faithfulness metrics and human-perceived qualities such as readability, usefulness, and trust.

1.3 Aim, Objectives and Research Questions

Aim: The core aim of this research is to design, develop, and evaluate an innovative automated classification and interpretation solution. This solution combines SetFit's superior classification performance under extremely imbalanced and few-shot conditions with the natural language generation capabilities of LLMs. The ultimate goal is to bridge the gap between the efficient computation of black-box deep learning models and the need for readable, reliable interpretations for software engineers.

Objectives: To achieve the above overall aim, this research will progressively implement the following four specific research and evaluation objectives:

1. To evaluate the classification performance of SetFit on imbalanced SBR datasets.
2. To compare LIME and SHAP in terms of explanation faithfulness using metrics.
3. To design a constrained prompt template for transforming feature attribution outputs into natural language explanations.
4. To evaluate whether LLM-assisted explanations improve readability, usefulness, and trust compared with keyword-based explanations.

⁴<https://ambari.apache.org/>

⁵<http://camel.apache.org>

⁶<https://db.apache.org/derby/>

⁷<https://wicket.apache.org>

Research Questions:

RQ1: How do explainable techniques (LIME and SHAP) compare in terms of faithfulness when explaining the predictions of the SetFit model for security bug reports?

RQ2: To what extent does the natural language explanation generated by LLM improve readability, usefulness, and trust compared to a simple LIME/SHAP keyword list?

1.4 Contributions

This thesis makes the following contributions:

- A pipeline that combines SetFit-based SBR classification, LIME/SHAP-based feature attribution, and LLM-based narrative generation.
- A comparison of LIME and SHAP using faithfulness metrics such as Log-Odds [15], Comprehensiveness [3], and Sufficiency [3].
- A human evaluation of whether natural language explanations improve perceived coherence, usefulness, and trust compared with keyword-based explanations.

1.5 Thesis Outline

Chapter 2 reviews related work on SBR classification, explainable AI, LIME, SHAP, and LLM-assisted explanation generation. Chapter 3 presents the research method, including the datasets, preprocessing steps, SetFit model configuration, explanation methods, faithfulness metrics, LLM prompt design, and user study setup. Chapter 4 reports the experimental results and analysis, including classification performance, explanation faithfulness, and user study outcomes. Chapter 5 discusses the findings in relation to the research questions, previous work, validity threats, and practical implications. Chapter 6 concludes the thesis by summarizing the main findings, outlining the contributions, and identifying directions for future work.

2.1 Security Bug Report Prediction

Security bug report prediction aims to identify whether a textual bug report describes a security-related issue. This task is important because SBRs may describe vulnerabilities that require a higher fixing priority than ordinary defects. A recurring problem in SBR prediction is that security-related reports may be mislabeled as non-security reports, partly because reporters do not always have sufficient security expertise [7]. Since manually inspecting large numbers of reports is time-consuming and often infeasible, automated SBR prediction has been proposed as a way to support security engineers during triage.

Because bug reports consist of natural language fields such as titles and descriptions, early SBR prediction studies commonly treated the task as a text mining or text classification problem [7, 8]. In an early industrial case study, Gegick et al. [7] trained a natural-language statistical model on Cisco bug reports and showed that text mining could help identify SBRs hidden among reports originally labeled as non-security. Goseva and Tyo [8] extended this line of work by comparing supervised and unsupervised classification methods. Their results suggest that supervised methods generally perform better, although they require labeled training data. Peters et al. [13] identified the problem of security cross words, where security-related terms occur in both SBRs and non-SBRs.

Another important challenge is label correctness. Wu et al. [20] showed that publicly available SBR datasets contain many mislabeled reports, and that correcting these labels substantially improves classification performance. Class imbalance is another structural difficulty in SBR prediction, since security reports usually form only a small minority of all bug reports. This imbalance can reduce classifier effectiveness and make accuracy an unreliable evaluation metric. Zheng et al. [21] demonstrated that class rebalancing methods can improve SBR classification performance in many settings. However, this line of work remains primarily focused on predictive performance rather than explaining model decisions. Recent work [6] has also explored the use of large language models. When faced with the highly imbalanced nature of SBR datasets, these models tend to overfit the majority class. Their performance in identifying SBRs is lower than that of traditional machine learning techniques [6].

Existing SBR prediction studies have mainly focused on improving classification performance under data imbalance and label noise. However, less attention has been paid to whether the resulting predictions can be explained in a way that is both faithful to the model and understandable to software practitioners. This motivates

the need to combine SBR prediction with post-hoc explanation and human-oriented explanation presentation.

2.2 Few-shot Learning and SetFit for SBR Prediction

A major limitation of traditional SBR prediction methods is their reliance on sufficiently large labeled datasets. In practice, labeled security bug reports are scarce and imbalanced, which makes few-shot learning a relevant direction for this task. SetFit [19] addresses few-shot text classification without relying on prompts or very large language models. Empirical results from the original SetFit study [19] show that the method can achieve competitive few-shot performance with fewer parameters and lower training cost than several prompt-based alternatives. Thus, Laiq [10] proposed using SetFit to overcome the limitations mentioned above, achieving significantly better classification performance. However, achieving high classification performance is not sufficient for practical adoption in security domains. Security analysts require transparent and interpretable reasoning to trust the model’s automated predictions. This lack of transparency necessitates the integration of explainability aspects to bridge the gap between high predictive accuracy and trust.

2.3 Explainable AI for Text Classification and LIME & SHAP

Explainable artificial intelligence (XAI) aims to make the behaviour of machine learning models more understandable to human users. In text classification tasks, explanations are commonly used to identify which words, tokens, or phrases contribute most strongly to a model’s prediction. This is particularly relevant for security bug report prediction, because developers and security analysts need to know not only whether a report is classified as security-related, but also which textual evidence supports that decision. Without such evidence, a high-performing classifier may still be difficult to trust or adopt in practice, especially in security-sensitive triage settings.

XAI methods can generally be divided into intrinsic and post-hoc explanation approaches. Intrinsic methods are models whose internal structures are designed to be interpretable, such as linear models or rule-based classifiers. In contrast, post-hoc methods explain the behaviour of an already trained model after prediction. Since SetFit relies on sentence-transformer representations and a downstream classification head, its internal decision process is not directly transparent to end users. Therefore, this thesis adopts post hoc explanation methods to explain individual SetFit predictions without altering the underlying classifier.

Local Interpretable Model-agnostic Explanations (LIME) is a widely used post hoc explanation technique. LIME explains a single prediction by creating perturbed versions of the original input and observing how the model’s prediction changes. It then fits a simpler local surrogate model around the instance being explained and uses it to estimate the contribution of individual features [14]. In text classification,

these features are usually words or tokens. The output of LIME is therefore a ranked list of textual features with associated weights, indicating how strongly each feature supports or opposes the predicted class. This makes LIME suitable for this thesis because it can provide local evidence for why SetFit predicts a bug report as security-related.

SHapley Additive exPlanations (SHAP) is another influential post-hoc explanation method. SHAP is based on Shapley values from cooperative game theory and estimates each feature’s contribution by considering its marginal effect across different feature coalitions [11]. In text classification, SHAP can assign contribution scores to words, subwords, or tokens depending on the tokenizer and explainer configuration. Compared with LIME, SHAP provides a more theoretically grounded attribution framework, but its output can be less straightforward to present to users when explanations are produced at the subword or tokenizer level. This is important in the context of this thesis, because the final explanation is intended not only for technical evaluation but also for human-oriented presentation.

2.4 Faithfulness, Plausibility, and Explanation Evaluation

Evaluating the quality of generative output is challenging. A critical risk when integrating LLMs into interpretability pipelines is their tendency to generate highly convincing text that may not accurately reflect the underlying model’s actual decision, leading interpretability evaluation to conflate faithfulness and plausibility; we should tease apart the two definitions [9]. Thus, this study academically distinguishes and defines the following core dimensions of technical terms [9]:

- **Faithfulness:** Faithfulness refers to the extent to which an explanation accurately and honestly represents the actual computation and reasoning process of the underlying black-box model (such as SetFit) when making a specific classification prediction. A highly faithful explanation must objectively reflect which input features truly drive the model’s decisions, regardless of whether the explanation appears logical to humans.
- **Plausibility:** Plausibility refers to how convincing, reasonable, and logically coherent the explanation appears to human users. LLMs are perfectly good at generating seemingly highly plausible explanations and may even inadvertently mask the model’s actual reasoning errors, but this does not equate to revealing the model’s true mechanism.
- **Readability & Usefulness:** Readability measures how easy it is for humans to read and understand the concepts in the explanation. Usefulness focuses on the practical utility of the explanation in real software engineering tasks and on improving system trust.

2.5 LLM-assisted Explanation Generation

Recent surveys have begun to explore the evolving intersection between xAI and LLMs. An earlier survey by Cambria et al. [2] investigated presentation methods in xAI, focusing on natural language explanations and reviewing 70 papers published up to 2021. They proposed a roadmap and a multi-criteria decision-making model to help select xAI methods based on their natural language presentation capabilities. Their review primarily covers earlier Natural Language Generation (NLG) techniques, rather than the recent focus on LLMs as dedicated narrative generators. More recently, Bilal et al. [1] provide a broad categorization of approaches, covering post-hoc explanations, human-centered narratives, evaluation metrics, benchmark datasets, and applications. However, it mixes two concepts: methods where LLMs generate explanations directly from raw data, and those where LLMs serve as translators that convert existing explanations into user-centered narratives. Thus, relying solely on LLMs, even with contextual prompting, is insufficient. From a critical standpoint, Suh et al. [17] challenge the premise of using LLMs for narrative translation, noting that such processes may obscure inconsistencies. In contrast, Matteo et al. [16] present the first focused synthesis of approaches that employ LLMs as post-hoc narrative generators—systems that articulate, contextualize, and communicate the outputs of established explainers rather than acting as explainers themselves. Through this effort, this research establishes the conceptual and methodological foundations needed to advance narrative-based explainability with LLMs.

2.6 Summary and Research Gap

The reviewed literature shows that security bug report prediction has been widely studied as a text classification problem. Prior work has demonstrated that textual information in bug reports can be used to identify security-related issues, and that challenges such as class imbalance, mislabeled reports, and security cross words can significantly affect classification performance. More recent work has also explored few-shot and large-language-model-based approaches for SBR identification. However, most existing studies remain primarily performance-oriented, focusing on metrics such as precision, recall, F1-score, MCC, and AUC. Although these metrics are important for evaluating predictive effectiveness, they do not explain why a model classifies a specific report as security-related.

Based on the reviewed literature, three main research gaps can be identified. First, existing SBR prediction studies have primarily focused on classification performance and paid limited attention to explaining individual predictions. Second, LIME and SHAP have not been sufficiently compared with respect to faithfulness in SetFit-based SBR prediction. Third, although LLMs may improve the readability and usefulness of technical explanations, there is limited evidence that attribution-grounded LLM narratives enhance human-perceived explanation quality without obscuring the distinction between faithfulness and plausibility.

This thesis addresses these gaps by evaluating a pipeline that combines SetFit-based SBR classification, post hoc explanations with LIME and SHAP, faithfulness-oriented evaluation metrics, and constrained LLM-based narrative generation. The

study first compares LIME and SHAP using objective faithfulness metrics, and then evaluates whether LLM-generated narratives improve perceived readability, usefulness, and trust compared with keyword-based attribution explanations. In this way, the thesis connects predictive performance, technical explanation faithfulness, and human-oriented explanation presentation in the context of security bug report prediction.

3.1 Research Design

This thesis adopts a controlled experimental design supported by a notebook-first implementation pipeline combined with a human evaluation. The experimental part is used to evaluate SetFit’s classification performance and to compare the faithfulness of LIME and SHAP explanations. The human evaluation assesses whether LLM-generated natural language explanations improve perceived readability, usefulness, and trust. This combined design is suitable because the thesis investigates both the quality of technical explanations and their utility.

This research process consists of four main phases, which are listed below and shown in Figure 3.1:

1. The SBR datasets (Ambari, Camel, Derby, Wicket) are collected and preprocessed
2. A SetFit model (sentence-transformer/all-mpnet-base-v2) is trained and evaluated using cross-validation.
3. Predictions classified as SBRs are explained using LIME and SHAP, and the resulting feature attributions are evaluated using faithfulness metrics.
4. The attribution outputs are transformed into natural language explanations using an LLM and evaluated through a user study.

3.2 Research Questions and Evaluation Strategy

The methodological design is directly aligned with the two research questions. RQ1 is addressed by comparing LIME and SHAP using objective faithfulness metrics after they explain the same SetFit predictions. RQ2 is addressed by comparing keyword-based explanations with LLM-generated narratives in a user study. This is necessary because an explanation can be technically faithful without being easy to understand, or easy to understand without being faithful. Table 3.1 describes the mapping relation between research questions and methods.

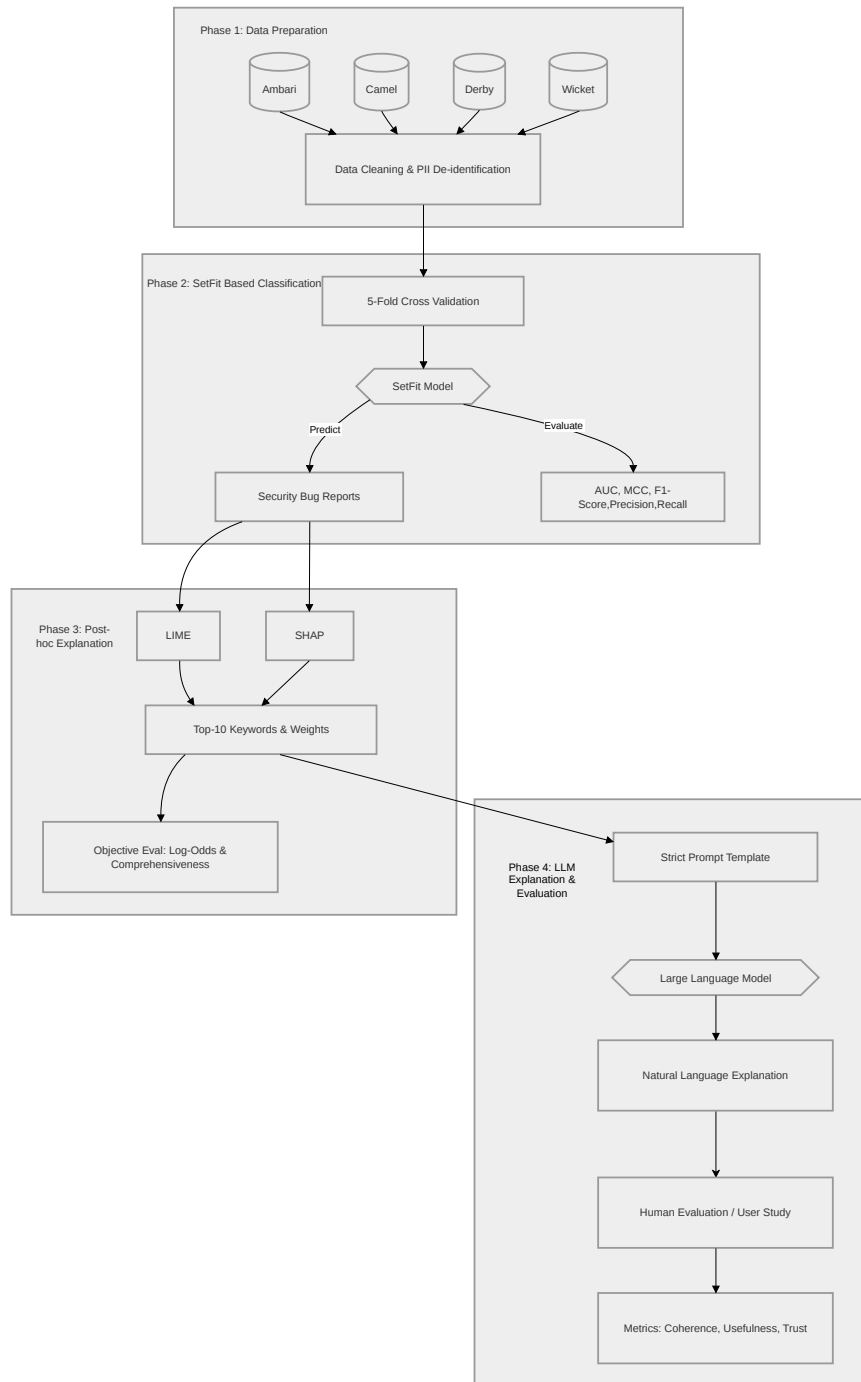


Figure 3.1: Overview of the research design and evaluation pipeline

Research Question	Input	Method	Output	Evaluation
RQ1	SetFit predictions on SBR datasets	LIME and SHAP explanations	Top-10 tokens and weights	Log-Odds, Comprehensiveness, Sufficiency
RQ2	Keyword explanations and LLM narratives	User study	Participant ratings	Coherence, usefulness, trust

Table 3.1: RQ-method mapping

3.3 Dataset Selection and Preprocessing

This study uses four open-source datasets: Ambari, Camel, Derby, and Wicket. These datasets have been used in prior SBR prediction research [10] and provide labelled bug reports suitable for security and non-security classification. They are selected because they represent realistic open-source software repositories and contain the class imbalance commonly observed in SBR identification. For each dataset, the original train and test CSV files are loaded and merged into a single dataframe while preserving the original split information as a provenance column. The classification label is the binary security field, where positive instances represent SBRs and negative instances represent non-SBRs. Table 3.2 shows the basic structure of dataset input.

Element	Description
Datasets	Ambari, Camel, Derby, Wicket
Raw files	Train_<Dataset>.csv, Test_<Dataset>.csv
Main text column	combined_text
Cleaned text column	cleaned_text
Label column	security
Fold column	fold

Table 3.2: Dataset input structure

Text preprocessing was designed to reduce noise while preserving security-related information. The cleaning function replaces email addresses, IP addresses, URLs, Jira ticket identifiers, user handles, phone-like strings, hexadecimal memory addresses, and Java stack-trace frames with placeholder tokens. English stop words are removed using `gensim`¹. Broad personal-name replacement is not applied because regex-based name detection may incorrectly remove technical identifiers in source-code-related text. This policy balances privacy protection with the need to retain meaningful technical evidence.

After cleaning, each dataset is assigned stratified five-fold splits using the security label. Stratification is used to preserve the minority SBR class distribution across folds, which is important because the datasets are imbalanced.

¹<https://pypi.org/project/gensim/>

3.4 SetFit-based SBR classification

SetFit was used as the SBR classifier because it is suitable for few-shot text classification scenarios. The baseline uses ‘sentence-transformers/all-mpnet-base-v2’ as the sentence encoder. The same training configuration is applied across all datasets: 5 folds, batch size 4, 10 contrastive learning iterations, 1 training epoch, and random seed 42. Keeping these settings fixed reduces configuration-driven variation when comparing explanation methods. For each report, the model produced a predicted label and a probability score for the SBR class, which were later used for explanation and faithfulness evaluation.

3.5 Post-hoc Explanation with LIME and SHAP

LIME and SHAP were applied to the same SetFit positive predictions to extract the top-10 influential textual features for the SBR class. Since text explainers may produce tokens at different levels of granularity, token normalization was applied before comparison. For SHAP, raw tokenizer-level outputs were preserved, while an additional word-level post-processing step was used to create more readable explanations for the LLM materials.

3.6 Faithfulness Evaluation Metrics

To answer RQ1, the faithfulness of LIME and SHAP explanations was evaluated by measuring how the SetFit model’s predicted probability changed when the tokens identified by each explainer were removed or retained. The underlying assumption is that a faithful explanation should identify textual features that are influential for the model’s decision. Therefore, if the top-ranked rationale tokens are removed from the input text, the model’s confidence in the original prediction should decrease. Conversely, if only the top-ranked rationale tokens are retained, a sufficient explanation should preserve the model’s confidence to some extent.

The evaluation used three faithfulness-oriented metrics: Log-Odds Drop [15], Comprehensiveness [3], and Sufficiency [3]. These metrics were calculated for the same set of positive SetFit predictions explained by LIME and SHAP. For each explanation, the top- k tokens were selected as the rationale, where $k = 10$ in this study. The same value of k was used for both explanation methods to ensure comparability. Let x denote the original bug report text, y denote the predicted class, and $p(y | x)$ denote the SetFit model’s predicted probability for class y given the original input. Let r denote the set of top- k rationale tokens selected by an explainer.

Before perturbing the text, rationale tokens were normalized to support robust matching between explanation outputs and the original text. The normalization converted tokens to lowercase and removed non-alphanumeric characters. For SHAP outputs, common transformer subword markers such as `##`, `\u0120`, and `\u2581` were removed before matching. This was necessary because SHAP may return tokenizer-level or subword-level units, whereas the perturbation procedure operates on normalized textual units.

For removal-based evaluation, a modified input $x_{\setminus r}$ was created by removing the normalized rationale tokens from the whitespace-tokenized original text. If all tokens were removed, the modified input was replaced with an [EMPTY] placeholder. The SetFit model was then applied again to this modified input. For retention-based evaluation, another modified input x_r was created by keeping only the normalized rationale tokens and removing all other tokens. This version was used to test whether the selected rationale tokens alone were sufficient to support the model’s original prediction.

The first metric is *Log-Odds Drop*. It measures the decrease in the log-odds of the predicted class after the rationale tokens are removed:

$$\text{LogOddsDrop}(x, r) = \log \frac{p(y \mid x)}{1 - p(y \mid x)} - \log \frac{p(y \mid x_{\setminus r})}{1 - p(y \mid x_{\setminus r})}. \quad (3.1)$$

A larger Log-Odds Drop indicates that removing the rationale tokens caused a stronger decrease in the model’s confidence. Therefore, higher values suggest that the selected tokens were more influential for the model’s prediction. To avoid numerical instability when probabilities were close to 0 or 1, probabilities were clipped to a bounded interval before computing the log-odds.

The second metric is *Comprehensiveness*. It measures how much the model’s predicted probability for the original class decreases when the rationale tokens are removed:

$$\text{Comprehensiveness}(x, r) = p(y \mid x) - p(y \mid x_{\setminus r}). \quad (3.2)$$

A higher comprehensiveness score indicates that the selected rationale tokens captured information that was important for the original prediction. If removing the selected tokens only slightly changes the prediction probability, then the explanation is less comprehensive because the highlighted tokens did not strongly affect the model’s output.

The third metric is *Sufficiency*. It evaluates whether the selected rationale tokens alone are enough to preserve the model’s original prediction:

$$\text{Sufficiency}(x, r) = p(y \mid x) - p(y \mid x_r). \quad (3.3)$$

Unlike Log-Odds Drop and Comprehensiveness, lower Sufficiency values indicate better explanation quality. A low sufficiency score means that the model’s confidence remains similar when only the rationale tokens are provided, suggesting that the selected tokens contain enough evidence to support the original prediction. A high sufficiency score indicates that the selected tokens alone are not sufficient to preserve the model’s confidence.

For each dataset, the three metrics were computed for all explained positive predictions and aggregated using the mean score. The comparison between LIME and SHAP was paired at the instance level because both explainers were applied to the same SetFit predictions. Log-Odds Drop and Comprehensiveness were interpreted as higher-is-better metrics, while Sufficiency was interpreted as a lower-is-better metric.

This evaluation focuses on the relationship between the attribution output and the behaviour of the SetFit model. It does not assume that the highlighted tokens necessarily correspond to the evidence that a human security expert would use.

Instead, the metrics assess whether the tokens selected by LIME or SHAP are important to the model under removal and retention tests. This distinction is important because RQ1 concerns technical faithfulness, whereas human-perceived explanation quality is evaluated separately in the user study.

3.7 LLM-based Explanation Generation

The LLM was used only as a narrative generator. It received the model-visible report text and attribution evidence, with LIME treated as the primary evidence and SHAP as optional diagnostic support. The prompt constrained the model to use only the provided text and attribution outputs, and explicitly prohibited unsupported claims about CVEs, exploit paths, root causes, or fixes. A local grounding audit were applied before storing the generated explanation.

3.8 User Study

A user study was designed to address RQ2 by evaluating whether LLM-generated natural language explanations improve human-perceived explanation quality relative to keyword explanations. The study focused on perceived usefulness and trust. These dimensions were selected because the goal of LLM-assisted explanation generation is to improve how developers understand and assess model predictions, rather than to replace objective faithfulness evaluation.

Each participant evaluated both types of explanations: a keyword-based explanation and an LLM-generated natural-language explanation. The keyword-based explanation consisted of the top attribution tokens and weights produced by the explainer (LIME here). The LLM-generated explanation presented the same attribution evidence in a short natural language narrative.

Participants rated each explanation using Likert-scale questions. The questionnaire measured two dimensions: usefulness and trust. Usefulness assessed whether the explanation helped participants interpret the model prediction. Trust assessed whether the explanation affected participants' confidence in the model prediction. The rating scale ranged from 1 to 5, with 1 indicating strong disagreement and 5 indicating strong agreement.

3.9 Data Analysis Procedure

The data analysis was organized according to the two research questions. For RQ1, the analysis focused on the technical faithfulness of LIME and SHAP explanations. For RQ2, the analysis focused on human-perceived explanation quality in the user study.

For the classification stage, SetFit performance was evaluated using AUC, MCC, precision, recall, and F1-score. These metrics were selected because SBR prediction is an imbalanced binary classification task. Accuracy was not used as a primary metric because it can be misleading when the non-security class dominates the dataset. AUC was used to assess ranking performance, while MCC, precision, recall, and

F1-score were used to evaluate threshold-dependent classification behaviour. The metrics were computed separately for each dataset and then summarized to provide the predictive context for the explanation analysis.

For RQ1, LIME and SHAP were applied to the same set of positive SetFit predictions. For each explanation, the top-10 rationale tokens were extracted and used to construct perturbed inputs. The three faithfulness metrics, Log-Odds Drop, Comprehensiveness, and Sufficiency, were then computed for each explained instance. The results were aggregated by dataset and explanation method using mean scores. Since LIME and SHAP explained the same predictions, their comparison was treated as paired at the instance level.

For RQ2, user study responses were analyzed descriptively. The Likert-scale ratings for usefulness and trust were summarized as means and standard deviations for each explanation format. The keyword-based explanations and LLM-generated explanations were then compared for each dimension. If sufficient responses were available, paired statistical tests were used to compare ratings between the two formats because the same participants evaluated both conditions. If the number of participants was small, the analysis emphasized descriptive trends rather than strong statistical claims.

Open-ended comments, if provided, were analyzed qualitatively to identify recurring observations about the strengths and weaknesses of each explanation format. Comments were grouped by whether they concerned readability, usefulness, trust, missing information, excessive confidence, or difficulty interpreting attribution weights. These qualitative observations were used to support the interpretation of the quantitative ratings.

3.10 Validity and Reliability Considerations

Several validity and reliability issues were considered when designing the study. Since this thesis combines machine learning classification, post-hoc explanation, LLM-based narrative generation, and human evaluation, threats may arise at different stages of the research pipeline.

Construct validity concerns whether the selected measures accurately represent the concepts being studied. For RQ1, faithfulness was measured using Log-Odds Drop, Comprehensiveness, and Sufficiency. These metrics capture how the SetFit model’s prediction changes when rationale tokens are removed or retained. However, they are still approximations of faithfulness and may not fully represent the internal reasoning of the model. To reduce this threat, multiple faithfulness metrics were used instead of relying on a single measure. For RQ2, readability, usefulness, coherence, and trust were measured using Likert-scale ratings. These ratings capture participant perceptions, but they do not prove that an explanation is objectively faithful or correct. Therefore, human-perceived quality was kept separate from technical faithfulness.

Internal validity concerns factors that may affect the observed results beyond the intended experimental variables. LIME and SHAP may be sensitive to parameter choices, tokenization, and perturbation strategies. To reduce this threat, the same top- k setting was used for both methods, and both explainers were applied

to the same SetFit predictions. In addition, token normalization was applied before perturbation to make removal and retention operations more consistent across explanation outputs. Another internal validity threat is that LLM-generated explanations may introduce unsupported information. This was mitigated through constrained prompt design and a grounding audit before using the generated explanations in the user study.

External validity concerns the generalizability of the findings. This thesis uses four open-source SBR datasets: Ambari, Camel, Derby, and Wicket. Although these datasets have been used in previous SBR prediction research and represent realistic open-source bug reports, they do not cover all software domains, reporting styles, or security contexts. Therefore, the results should not be generalized to all software projects without further evaluation.

Conclusion validity concerns the strength of the conclusions drawn from the data. The number of explained samples, the number of user study participants, and the variability across datasets may affect the robustness of the findings. To reduce this threat, results were reported separately by dataset where possible, and the discussion avoids overgeneralizing from a single metric or dataset. If the user study sample size is small, the analysis emphasizes descriptive patterns rather than strong statistical claims.

Reliability concerns whether the study can be repeated with similar procedures and obtain comparable results. To improve reliability, the experimental pipeline used fixed random seeds where possible, consistent preprocessing rules, the same SetFit configuration across datasets, and the same top-10 rationale setting for both LIME and SHAP. The helper functions used for classification metrics, token normalization, text perturbation, and faithfulness calculation were implemented as reusable scripts. The complete prompt template, user study material, and additional result tables are provided in the appendix to support transparency and reproducibility.

4.1 Overview of the Results

This chapter presents the results obtained from the experimental pipeline. First, the datasets and preprocessing outputs are summarized to describe the empirical basis of the study. Second, the SetFit classification results are reported to establish the predictive context for the explanations generated. Third, LIME and SHAP explanation outputs are analyzed and compared using faithfulness metrics. Finally, the LLM-generated narratives and human-oriented comparison materials are presented to support the evaluation of the coherence, usefulness, and trustworthiness of explanations.

4.2 Dataset and Preprocessing Results

The study uses four open-source SBR datasets: Ambari, Camel, Derby, and Wicket. Each dataset was preprocessed using the cleaning procedure described in Chapter 3. The preprocessing step produced a cleaned text field used for SetFit classification and subsequent explanation generation. The cleaning procedure removed or replaced potentially noisy or sensitive patterns, such as email addresses, URLs, IP addresses, user handles, phone-like strings, and stack trace frames, while preserving security-relevant textual evidence.

4.3 SetFit Classification Results

Table 4.1 presents the SetFit classification results for the four datasets. The results are reported using AUC, MCC, precision, recall, and F1-score. Overall, the performance varies across datasets, indicating that SBR prediction difficulty is not uniform across projects. These results provide the prediction context for the subsequent explanation and faithfulness analysis.

After classification, reports predicted as security-related were selected for explanation generation. There are 279 samples in total.

4.4 LIME and SHAP Explanation Results

For each selected positive prediction, LIME and SHAP generated ranked lists of influential textual features for the SBR class. Each explanation consists of the top-

Dataset	AUC	MCC	Precision	Recall	F1-score
Ambari	0.7766	0.2629	0.4410	0.1970	0.2641
Camel	0.8454	0.6891	0.8900	0.5676	0.6850
Derby	0.8922	0.7717	0.8430	0.7822	0.8088
Wicket	0.8721	0.8083	0.8992	0.7444	0.8106

Table 4.1: SetFit classification results

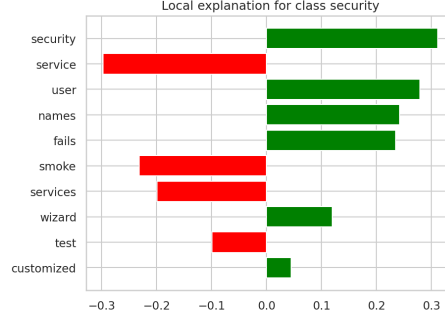


Figure 4.1: LIME explanation

10 tokens and their associated weights. These token-weight pairs serve as both the input for faithfulness evaluation and the evidence used for LLM-assisted narrative generation.

The examples 4.1 4.2 illustrate that both methods highlight security-relevant terms, but their rankings and tokens may differ.

A practical difference between LIME and SHAP was the granularity of extracted features. SHAP sometimes produced tokenizer-level units, which are less suitable for human-oriented presentation. Therefore, the raw SHAP outputs were preserved for comparison, while an additional word-level aggregated version was used when preparing readable explanation materials.

4.5 Faithfulness Evaluation Results

Figure 4.3 visualizes the faithfulness scores for LIME and SHAP. Log-Odds drop and Comprehensiveness are interpreted as higher-is-better metrics, because larger decreases in SBR probability indicate that the highlighted tokens were influential.

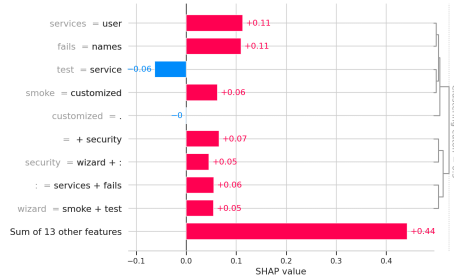


Figure 4.2: SHAP explanation

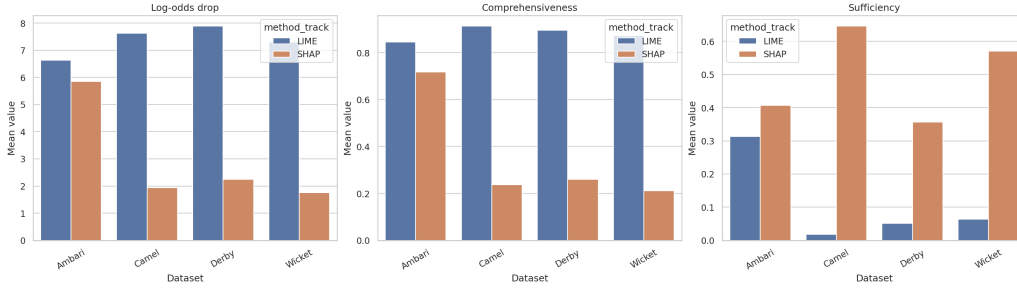


Figure 4.3: Faithfulness metrics results

Table 4.2: Grounding audit results for LLM-generated explanations

Grounding status	Number of explanations	Percentage
PASS	273	97.85%
VIOLATION	6	2.15%
Total	279	100.00%

Sufficiency is interpreted in the opposite direction: lower values indicate that the selected tokens are sufficient to preserve the model’s prediction.

LIME obtains a higher average Log-Odds drop and Comprehensiveness than SHAP, suggesting that its highlighted tokens more strongly affect the SetFit prediction.

Based on the faithfulness metrics, RQ1 is answered by comparing how strongly LIME and SHAP rationales affect SetFit probabilities. The results suggest that LIME provides more faithful explanations across all datasets.

4.6 LLM-assisted Explanation Results

The LLM-assisted explanation stage transformed attribution-based keyword evidence into short natural language narratives. The LLM was used only as a narrative generator and not as an independent explainer. Each generated explanation was grounded in the model-visible report text and the attribution outputs. The purpose was to make the attribution evidence easier to understand while avoiding unsupported claims.

The LLM explanation file contains 279 generated explanations, corresponding to the same positive SetFit predictions used in the LIME and SHAP explanation stage. A local grounding audit was applied to check whether the generated explanations introduced unsupported claims. Table 4.2 summarizes the grounding audit results.

Most generated explanations passed the grounding audit. Six explanations were flagged as violations. The detected unsupported terms included cases such as *SQL*, *encryption*, *permission*, and *authorization*. These violations indicate that even when an LLM is constrained by attribution evidence, it may still introduce security-related concepts that are not sufficiently supported by the provided input. Therefore, the grounding audit is necessary before using generated narratives in a human evaluation.

4.7 Human Evaluation Results

At the time of this draft, the user study results are not yet available.

4.8 Summary of Findings

This chapter reports the current results of the SetFit classification, explanation extraction, faithfulness evaluation, and LLM-assisted explanation generation stages.

First, the SetFit classification results showed that model performance varied across datasets. Derby and Wicket achieved the strongest overall performance, while Ambari was the most challenging dataset. This variation is important because explanation quality must be interpreted in relation to the behaviour of the underlying classifier.

Second, LIME and SHAP produced attribution-based explanations for the same 279 positive SetFit predictions. The example analysis showed that both methods could identify relevant textual evidence, but SHAP sometimes produced outputs that were less directly suitable for human-facing presentation.

Third, the faithfulness evaluation showed that LIME outperformed SHAP across all four datasets under Log-Odds Drop, Comprehensiveness, and Sufficiency. These results provide the current answer to RQ1: in this experimental setting, LIME produced more faithful explanations of SetFit SBR predictions than SHAP.

Fourth, the LLM-assisted explanation stage generated natural language narratives for all 279 explained predictions. The grounding audit showed that 273 explanations passed and 6 were flagged as violations. This indicates that constrained prompting can produce mostly grounded explanations, but audit or filtering remains necessary because unsupported security-related terms may still appear.

5.1 Overview of the Main Findings

The thesis investigates explainable security bug report prediction by combining SetFit-based classification, post hoc explanation with LIME and SHAP, and LLM-assisted narrative generation. The study designed two research questions. RQ1 examined how LIME and SHAP compare in terms of faithfulness when explaining SetFit predictions for SBRs. RQ2 examined whether LLM-generated natural language explanations improved perceived coherence, usefulness, and trust compared with keyword-based attribution explanations.

5.2 Faithfulness of LIME and SHAP

The faithfulness results show that LIME performed better with SetFit’s prediction behaviour than SHAP under this experimental setting. Across Ambari, Camel, Derby, and Wicket, LIME produced higher Log-Odds Drop and Comprehensiveness scores than SHAP. This means that the tokens selected by LIME had a stronger effect on the SetFit model’s predicted probability when removed from the input text. In addition, LIME achieved lower Sufficiency scores across all datasets, which indicates that the LIME-selected tokens were more sufficient for preserving the original prediction when used alone.

One possible explanation for this result is that LIME’s perturbation-based mechanism aligns well with the removal-based faithfulness metrics used in this thesis. LIME explains predictions by perturbing the input and estimating the local contribution of textual features. The evaluation metrics also rely on perturbing the input by removing or retaining rationale tokens. Therefore, LIME’s explanation generation process may be more naturally compatible with the faithfulness evaluation design. This does not mean that LIME is always superior to SHAP, but it suggests that LIME was better suited to the perturbation-based evaluation framework used here.

Another possible reason is the granularity of the explanation outputs. LIME produced word-level features that could be directly matched and removed from the original text. SHAP, however, sometimes produced tokenizer-level or subword-level outputs. Although the thesis applied normalization and word-level post-processing for readability, SHAP’s raw outputs were not always as directly aligned with whitespace-tokenized text perturbation. This may have reduced the measured faithfulness of SHAP under the implemented evaluation procedure.

However, the conclusion that LIME was more faithful than SHAP should be interpreted within the scope of this thesis. The comparison depends on several implementation choices, including the top-10 rationale setting, token normalization, text removal strategy, and probability-based faithfulness metrics. A different SHAP configuration, a different text masking strategy, or a different model architecture might produce different results. Therefore, the findings should be understood as evidence that LIME was more faithful than SHAP for the studied SetFit-based SBR prediction pipeline, rather than as a universal claim about all text classification tasks.

5.3 LLM-generated Explanations and Human Perception

The generated explanations in this thesis were grounded in LIME/SHAP attribution evidence and the model-visible bug report text. The LLM was not asked to independently determine whether a report contained a real vulnerability. Instead, it was instructed to explain why the SetFit model may have predicted the report as security-related based on the provided attribution evidence. This design is important because it limits the role of the LLM to explanation presentation rather than explanation discovery.

5.4 Implications for Research and Practice

The findings have several implications for research on security bug report prediction.

First, the results suggest that classification performance alone is insufficient for evaluating SBR prediction models. A model may achieve strong AUC, MCC, or F1-score, but users still need to understand why individual reports are classified as security-related. This is especially important for security bug triage, where false positives can waste expert effort and false negatives can delay vulnerability mitigation.

Second, the thesis shows the importance of evaluating explanation faithfulness in SBR prediction. Post-hoc explainers can produce plausible-looking token rankings, but their quality should be tested against the model’s behaviour. The use of Log-Odds Drop, Comprehensiveness, and Sufficiency provides a practical way to evaluate whether highlighted tokens actually influence predictions. This type of evaluation can help avoid treating explanation outputs as reliable simply because they contain security-relevant terms.

Third, the comparison between LIME and SHAP suggests that explainer choice matters. Even when two explainers are applied to the same model and the same predictions, they may produce different attributions and different faithfulness scores. Researchers should therefore avoid assuming that all post-hoc explanation methods behave similarly. Instead, explanation methods should be evaluated in the specific model, dataset, and task context in which they are used.

Fourth, the LLM-assisted explanation stage shows that LLMs may play a useful role as explanation presenters. They can transform technical attribution outputs into more readable narratives. However, the LLM should not be treated as an independent source of explanation. In this thesis, the LLM was constrained to use

only the provided attribution evidence and report text. This design helps maintain a connection between the narrative and the model explanation, while reducing the risk of hallucinated security claims.

5.5 Threats to Validity

As discussed in Chapter 3.10

5.6 Broader Discussion

The broader implication is that explainable SBR prediction requires a balance between technical faithfulness and human readability. Faithfulness metrics are necessary to evaluate whether explanations reflect model behaviour, while user studies are necessary to determine whether explanations are understandable and useful. A combined explanation format that presents both attribution evidence and grounded natural language narratives may offer a practical direction for future explainable SBR triage tools.

6.1 Conclusion

This thesis investigated explainable security bug report prediction by combining SetFit-based classification, post-hoc explanation methods, and LLM-assisted natural language explanation generation. The motivation for the study was that prior SBR prediction research has mainly focused on classification performance, while paying less attention to whether individual predictions can be explained in a way that is both faithful to the model and understandable to users.

6.2 Future Work

Several directions for future work can extend the findings of this thesis.

Future research should explore additional explanation methods. This thesis compares LIME and SHAP, but other post-hoc explanation techniques could also be evaluated. Including more explainers would provide a broader understanding of which methods are most suitable for explaining SBR predictions.

References

- [1] A. Bilal, D. Ebert, and B. Lin, “Llms for explainable ai: A comprehensive survey,” *arXiv preprint arXiv:2504.00125*, 2025.
- [2] E. Cambria, L. Malandri, F. Mercorio, M. Mezzanzanica, and N. Nobani, “A survey on xai and natural language explanations,” *Information Processing & Management*, vol. 60, no. 1, p. 103111, 2023.
- [3] J. DeYoung, S. Jain, N. F. Rajani, E. Lehman, C. Xiong, R. Socher, and B. C. Wallace, “Eraser: A benchmark to evaluate rationalized nlp models,” in *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, pp. 4443–4458.
- [4] R. Dwivedi, D. Dave, H. Naik, S. Singhal, R. Omer, P. Patel, B. Qian, Z. Wen, T. Shah, G. Morgan *et al.*, “Explainable ai (xai): Core ideas, techniques, and solutions,” *ACM computing surveys*, vol. 55, no. 9, pp. 1–33, 2023.
- [5] K. Erdil, E. Finn, K. Keating, J. Meattle, S. Park, and D. Yoon, “Software maintenance as part of the software life cycle,” *Comp180: Software Engineering Project*, vol. 1, pp. 1–49, 2003.
- [6] H. L. França, K. Goseva-Popstojanova, C. Teixeira, and N. Laranjeiro, “Gpts are not the silver bullet: Performance and challenges of using gpts for security bug report identification,” *Information and Software Technology*, vol. 185, p. 107778, 2025.
- [7] M. Gegick, P. Rotella, and T. Xie, “Identifying security bug reports via text mining: An industrial case study,” in *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. IEEE, 2010, pp. 11–20.
- [8] K. Goseva-Popstojanova and J. Tyo, “Identification of security related bug reports via text mining using supervised and unsupervised classification,” in *2018 IEEE International conference on software quality, reliability and security (QRS)*. IEEE, 2018, pp. 344–355.
- [9] A. Jacovi and Y. Goldberg, “Towards faithfully interpretable nlp systems: How should we define and evaluate faithfulness?” in *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, pp. 4198–4205.
- [10] M. Laiq, “Few-shot learning for security bug report identification,” *arXiv preprint arXiv:2601.02971*, 2026.
- [11] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” *Advances in neural information processing systems*, vol. 30, 2017.

- [12] A. Paleyes, R.-G. Urma, and N. D. Lawrence, “Challenges in deploying machine learning: a survey of case studies,” *ACM computing surveys*, vol. 55, no. 6, pp. 1–29, 2022.
- [13] F. Peters, T. T. Tun, Y. Yu, and B. Nuseibeh, “Text filtering and ranking for security bug report prediction,” *IEEE Transactions on Software Engineering*, vol. 45, no. 6, pp. 615–631, 2017.
- [14] M. T. Ribeiro, S. Singh, and C. Guestrin, “" why should i trust you?" explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.
- [15] A. Shrikumar, P. Greenside, and A. Kundaje, “Learning important features through propagating activation differences,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 3145–3153. [Online]. Available: <https://proceedings.mlr.press/v70/shrikumar17a.html>
- [16] M. Silvestri, V. Vineis, E. Gabrielli, F. Giorgi, F. Veglianti, F. Silvestri, and G. Tolomei, “A survey on explainable ai narratives based on large language models,” *Authorea Preprints*, 2025.
- [17] A. Suh, K. Alperin, H. Li, and S. R. Gomez, “Don’t just translate, agitate: Using large language models as devil’s advocates for ai explanations,” *arXiv preprint arXiv:2504.12424*, 2025.
- [18] U. B. Torun, M. T. Demircan, M. F. Gön, and E. Tüzün, “Past, present, and future of bug tracking in the generative ai era,” *ACM Trans. Softw. Eng. Methodol.*, 2026. [Online]. Available: <https://doi.org/10.1145/3806655>
- [19] L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, and O. Pereg, “Efficient few-shot learning without prompts,” *arXiv preprint arXiv:2209.11055*, 2022.
- [20] X. Wu, W. Zheng, X. Xia, and D. Lo, “Data quality matters: A case study on data label correctness for security bug report prediction,” *IEEE Transactions on Software Engineering*, vol. 48, no. 7, pp. 2541–2556, 2021.
- [21] W. Zheng, Y. Xun, X. Wu, Z. Deng, X. Chen, and Y. Sui, “A comparative study of class rebalancing methods for security bug report classification,” *IEEE Transactions on Reliability*, vol. 70, no. 4, pp. 1658–1670, 2021.

Appendix A

Supplemental Information

A.1 AI Declaration

Grammaly was used to improve the spelling of the thesis.

A.2 User Study Questionnaire

Link available online: [\[Click here\]](#)

A.3 LLM Prompt

System prompt: "You are a cautious explainability writer for software developers. " "Return one grounded summary that combines the issue text with the strongest LIME evidence and optional SHAP support. Do not add security facts that are not grounded in the report text or attribution evidence."

Hard constraints: "Summarize what the model-visible issue report appears to describe." "Briefly explain why the classifier likely marked it as security-related, using LIME as the primary evidence and SHAP only as optional support." "Write natural prose, not a token-by-token or weight-by-weight list." "Do not add facts absent from both the model-visible text and the attribution evidence." "Do not infer CVEs, exploit paths, real vulnerability types, root causes, or fixes unless they are explicitly present." "Prefer wording such as 'appears to concern', 'is framed around', or 'the model likely treats this as' over unsupported factual causality such as 'is caused by'." "If evidence is weak, mixed, or truncated, say so explicitly." "Return only valid JSON with the requested schema."

